Coordinating Binary Trait: Accurate and Lightweight Runtime On-Chip Power Meter Design

Qiang Li, Weixi Lyu, Yifan Liu, Jun Tao, *Senior Member, IEEE*, and Jun Han, *Member, IEEE*

***Abstract*—As heterogeneous platforms scale, power management issues grow increasingly critical. On-chip power awareness is fundamental to effective power management on such platforms. However, the integration of diverse components leads to higher power consumption and more complex power patterns, complicating the perception of power distribution. To address this challenge, this article presents COordinating BInary Trait (COBIT), which focuses on the design and optimization of on-chip power meters (OPMs) for heterogeneous design, enabling power management in large-scale systems. COBIT extracts essential bitwise wires as features, learns a boosting model based on binary trees, and implements the OPM for runtime power prediction. Additionally, multiobjective optimization algorithms are employed in the design space exploration of the power meter to ensure an accurate and low-overhead integration. Experimental results demonstrate the capabilities of COBIT on heterogeneous hardware. Using only 0.028% bitwise power proxies of all RTL wires in the NVIDIA deep learning accelerator (NVDLA) and 0.036% in the Berkeley out-of-order machine (BOOM) processor core, the per-cycle models achieve a mean absolute percentage error (MAPE) of 1.69% and 2.49%, respectively. Regarding the success rate of predictions on power peaks in NVDLA, COBIT achieves a maximum performance improvement of 12.93% and an average improvement of 8.11% over existing methods. Moreover, the gate-area/power overhead of our OPM on BOOM and NVDLA is 1.25%/0.217% and 0.49%/0.097%, respectively, while performing per-cycle power prediction in just three cycles. Unlike previous approaches, which struggle with balancing accuracy and efficiency, COBIT effectively addresses both challenges, delivering unprecedented benefits for large-scale systems.**

***Index Terms*—Design space exploration, heterogeneous platform, modeling, on-chip power meter (OPM), power management.**

NOMENCLATURE

$\mathcal{D}$	Dataset in power modeling.
N	Number of cycles.
M	Number of all wires in design.
Q	Number of bit-level proxies selected in design.
$\mathbf{x}, \mathbf{x}_i$	Toggle matrix and toggle vector at each cycle.
$\hat{y}_i, y_i$	Power prediction and power label at per cycle.
p^t	Multicycle prediction.
λ, γ	Hyperparameters of LR-MCP algorithm.
R	Number of boosting rounds.
$\mathbf{W}$	Leaf score matrix of all trees.
$\boldsymbol{S}$	List of samplers $\{s_1, s_2, \cdots, s_n\}$.
$\boldsymbol{P}$	List of pruners $\{p_1, p_2, \cdots, p_n\}$.
$\boldsymbol{hp\_rgs}$	Design space of all the hyperparameters in building boosting trees during hyperparameter optimization.
pop_size	Population size of generation in evolutionary algorithms or the number of new candidate trials in the Bayesian optimization algorithm.
POF	Pareto optimal front with a set of Pareto optimal solutions.
hv	Hypervolume metric of a sampler and pruner pair $\{s, p\}$.
$\boldsymbol{pair\_hv}$	Collection of tuples of a pair and its corresponding hv.
CP	Candidate pair used for searching a power model for performance comparison.
$\boldsymbol{R\_rgs}$	Range of boosting round used for the HPO.
T_th	Threshold number of leaves in all binary trees.
BT	Best trial, i.e., hyperparameters chosen from a POF.

Received 16 January 2025; revised 16 June 2025, 23 July 2025, 25 July 2025, and 6 August 2025; accepted 14 August 2025. Date of publication 25 August 2025; date of current version 27 November 2025. This work was supported by the National Natural Science Foundation of China under Grant 61934002 and Grant 62234008. *(Corresponding author: Jun Han.)*

The authors are with the State Key Laboratory of Integrated Chips and Systems, School of Microelectronics, Fudan University, Shanghai 200433, China (e-mail: qiangli19@fudan.edu.cn; wxlv22@m.fudan.edu.cn; yifliu21@m.fudan.edu.cn; taojun@fudan.edu.cn; junhan@fudan.edu.cn).

Digital Object Identifier 10.1109/TVLSI.2025.3599872

I. INTRODUCTION

HETEROGENEOUS multiprocessor system-on-chip has been a foundational pillar in mobile computing platforms, desktop processors, and data centers over the past decades. Recently, the explosive growth of artificial intelligence computing has led to the emergence of heterogeneous platforms equipped with high-performance graphics processing units (GPUs) and neural processing units (NPUs) [9], [35].

Meanwhile, heterogeneous integration has offered a new opportunity to propel ultralarge-scale processor innovation after Moore's law. There are growing products from both industry and academia, such as Intel Ponte Vecchio and AMD Instinct MI300A [3], [22]. However, the integration of chiplets exacerbates power consumption issues. For instance, the latest NVIDIA Blackwell GPU GB200 reaches the thermal design power of 2700 W [6]. Furthermore, work [4] claims that

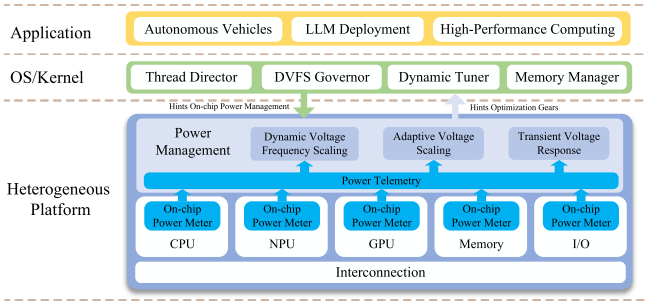

Fig. 1. Application of OPM in heterogeneous platforms.

the absolute power magnitude of heterogeneous integration processors is projected to escalate to 50 kW within the next decade.

As illustrated in Fig. 1, the increasing heterogeneity and integration of diverse components in large-scale systems underscore the critical need for on-chip power management techniques, such as dynamic voltage and frequency scaling (DVFS), adaptive voltage scaling (AVS), and transient voltage response (TVR). Various studies have explored intelligent agents and software–hardware codesign to improve the efficiency of such platforms [20], [31], [34], [40]. Notably, many intelligent power management approaches require power consumption telemetry and recognize it as a crucial system state. Hence, on-chip power meters (OPMs) are essential for optimizing the efficiency and performance of on-chip power management in large systems.

However, previous OPM designs lack consideration of the complex power patterns on heterogeneous designs, such as NPU and central processing unit (CPU). The patterns on NPU are significantly different from CPU.

Many NPUs are rated on the number of millions or even billions of multiply–accumulate operations per second. However, the operation easily induces glitches. As is known, wide fan-in and deep combinational logic imply a higher likelihood of glitches until it settles down. The situation is that data paths in an NPU could be as deep as tens of stages, and a glitch can propagate through the whole combinational circuit and cause wasted power consumption from every gate it passes. Remarkably, the glitch power dissipation accounts for up to 40% of total dynamic power in some designs [10]. Therefore, the complex power patterns of heterogeneous hardware deserve more attention during OPM design. Moreover, voltage noise represented by $L \times dI/dt$ quickly emerges during high-performance logic operations, sometimes leading to IR-Drop exceeding 1 kW [26]. This poses a significant challenge to the reliability and power integrity of ultralarge-scale processors.

Previous studies typically use event counters to monitor architectural behavior and estimate power consumption at runtime. However, the coarse-grained statistics collected over millions of profiling cycles overlook the activity variability within circuits occurring within few nanoseconds, resulting in imprecise power predictions at runtime. In contrast, recent advancements in fast power management [21] and voltage boosting technology [24] have imposed stringent demands on the response speed and accuracy of on-chip power prediction, exposing the limitations of these methods.

Recent works emphasize that machine learning-based power modeling at the register transfer level is a promising approach for OPM design. APOLLO and DEEP [44], [45] utilize linear models to predict power consumption based on RTL-level power proxies, multibit signals for the former, and single-bit wires for the latter. However, the predictors based on linear regression lack the accuracy needed for power peak prediction on NPUs. Simmani [27] employs a nonlinear polynomial model for power prediction. However, implementing this model on-chip incurs significant overhead due to the large memory required for storing long-window activity counters and the costly multiplication logic needed to realize polynomial terms. Decision tree and bagging tree methods for power modeling have also been proposed in [32] and [33], but they fail to provide accurate and fast responses due to their redundant feature formats and the heavy OPM design.

To meet the demands of power management and voltage noise mitigation in next-generation heterogeneous platforms, OPMs should prioritize: 1) high accuracy across diverse power patterns; 2) low inference latency for fine-grained control; and 3) small area and power overhead relative to the target design.

In this article, we propose COordinating BInary Trait (COBIT), which achieves an accurate and lightweight OPM design by coordinating bitwise power proxies with a nonlinear boosting model based on binary trees, to tackle the three challenges outlined above. Within this framework, OPMs with high accuracy, low latency, and small overhead are designed and optimized through hyperparameter optimization (HPO) for different target designs. Unlike most prior works that remain at the simulation or algorithmic level, COBIT is developed with hardware integration in mind, targeting direct RTL deployment. As a result, it is applicable not only to architectural exploration but also to practical on-chip implementation across diverse processor designs. The main contributions of this work are summarized as follows.

1) *Binary-Aware Codesign of Proxy and Model:* The bitwise proxy and the binary tree model share an inherent binary nature. This structural alignment allows our boosting-based OPM to fully exploit the synergy between input encoding and tree growth, resulting in high modeling efficiency with small hardware overhead and low inference latency.
2) *Cost-Aware Model Exploration Driven by Multiobjective Optimization:* A dedicated HPO framework is integrated to jointly optimize the prediction accuracy and hardware overhead of the proposed OPM. This approach eliminates manual design and tuning efforts, enabling an automated and scalable model exploration.
3) *Comprehensive Evaluation and Superior Performance on Peak Scenario:* COBIT is the first to systematically evaluate power prediction under peak scenarios by identifying statistical power surges and assessing prediction performance. The proposed method achieves accurate and reliable power tracking across heterogeneous designs, especially in NPUs where transient power events are more prominent.

4) *Robust Validation and OPM Implementation:* The methodology is validated on a heterogeneous platform that includes the Berkeley out-of-order machine (BOOM) RISC-V core and the NVIDIA deep learning accelerator (NVDLA). The prediction accuracy is analyzed across various power patterns, while the latency and overhead of our synthesizable OPM are verified using the TSMC 28-nm CMOS process.

The remainder of this article is organized as follows. Section II reviews the related work and clarifies the motivation of this article. Section III provides the methodology of the COBIT framework, while Section IV compares and discusses our results and state-of-the-art works. Section V gives the conclusion of this work.

II. Background

A. Literature Review

Power sensors in large-scale systems provide accurate power estimation at runtime; however, they lack fine-grained temporal resolution and impose significant area overhead. Recently, runtime power prediction through power modeling has been explored at both the architecture and RTL levels.

At the architecture level, performance counters are commonly designed and instrumented in modern processors. Numerous studies [28], [29], [39] have shown that performance counters offer acceptable accuracy for average-based power prediction. This approach is widely used to address dynamic power management problems in heterogeneous platforms [31], [34]. However, the design of performance events requires prior knowledge of the microarchitecture, which limits their automation for new designs. Additionally, this methodology is restricted to coarse-grained power management, as it estimates power across millions of simulation cycles and cannot provide accurate per-cycle power estimation.

At the RTL level, OPM requires circuit-level wires/signals as features to predict power consumption. The features are recognized as power proxies. Numerous studies have adopted the proxy-based approach to support design-time power analysis and implemented OPM to enable runtime power estimation [16], [32], [41], [44], [45]. As discussed before, OPMs are desired to achieve high accuracy, low latency, and small overhead. Additionally, power architects prefer simple feature engineering and minimal modeling effort during OPM design. PrEsto [41] employs linear models to characterize the ARM Cortex-A8 processor, utilizing extensive feature engineering to reduce the complexity of the power model, which needs heavy modeling effort and fails to provide nanosecond-level responses. APOLLO [45] employs the linear regression with minimax concave penalty (MCP) to automatically filter signal-level proxies from all RTL signals, followed by refitting a linear model to implement the OPM. However, the bits in signal-level proxies are redundant and result in extra overhead for feature routing and buffering. Furthermore, the power peaks that frequently emerge in NPUs are not considered or validated in their work. DEEP [44] simplifies signal-level proxies to bit-level proxies, indicating picking out bitwise wires from all RTL wires, which mitigates the processing burden of input features in the OPM. Additionally, it uses a best subset selection method to choose the highest performance subset from the candidate proxy set. However, the subset selection method is time-consuming, and the linear model struggles to adapt to complex power patterns in NPUs when fewer proxies are prioritized.

In contrast, Simmani [27] uses singular value decomposition to reduce the dimensionality of signal activity features and employs K-means++ to cluster the features. Subsequently, a polynomial regression model is proposed to predict power consumption. However, this approach requires extensive input feature processing and results in a complex, heavy OPM with slow response. Lin et al. [32] monitor the switching activity of candidate signals within microsecond-level sampling periods, also requiring the preprocessing of input features. Furthermore, the bagging tree structure is fully retained in memory, causing the execution time to reach up to 21 cycles for a single power estimation, which fails to meet the temporal resolution requirements for fast power management. Unlike existing tree-based OPMs [32], Lin et al. [33], COBIT, introduce a unique binary coordination between selected bitwise proxies and the binary structure of boosting trees. This structural alignment allows for highly efficient feature utilization and rapid inference. In contrast to bagging tree models with deep or unbalanced paths, our boosting trees are shallow and optimized via HPO, significantly reducing implementation overhead and enhancing prediction accuracy. Table I summarizes the key characteristics of representative OPM designs, highlighting how COBIT differs from prior methods in terms of input granularity, modeling strategy, temporal resolution, performance, and hardware efficiency.

COBIT offers several key observations regarding existing methods.

1) Previous linear modeling approaches, such as those proposed in [44] and [45], have struggled to achieve accurate predictions with fewer proxies, especially for power peaks. However, minimizing the number of proxies is highly desirable for efficient OPM implementation and scalability on larger heterogeneous platforms.
2) While the linear model simplifies OPM implementation, it fails to meet the stringent accuracy requirements for complex power patterns. In contrast, nonlinear models achieve higher accuracy but incur substantial implementation overhead. Therefore, is there a way to ensure low overhead while achieving high accuracy?
3) Furthermore, the hyperparameter design space in power model learning is inherently large and has a significant impact on both prediction accuracy and hardware overhead. However, prior works often overlook this complexity in two key aspects. First, most existing approaches adopt single-objective tuning, optimizing only for prediction accuracy without explicitly considering hardware implementation cost. For example, works such as [32] and [33] employ grid search and evaluate configurations solely based on accuracy. Xie et al. [44], [45] also rely on validation-set accuracy to guide their model selection, without jointly optimizing for costs. Likewise, Simmani [27] does not estimate

TABLE I
OPM DESIGN AND IMPLEMENTATION CHARACTERISTICS IN REPRESENTATIVE REGISTER TRANSFER-LEVEL PROXY-BASED METHODS

Works	Feature	Feature Selection	Model Type	Peak Evaluated?	DSE Used?	Time Res.	#Proxy*	Performance†	Hardware‡
TCAD'19 [32]	RTL Signals with high activity	Recursive elimination	Bagging decision tree	✗	✗	1000s cycles	1×/1×	2.95%/26%	-/≤1.22%/21
MICRO'19 [27]	ALL RTL signals	K-means++	Polynomial	✗	✗	100s cycles	6.7×/-	4.5%/-	N/A
MICRO'21 [45]	ALL RTL signals	LR-MCP	Ridge	✗	✗	Per-cycle	4.8×/-	2.24%/-	0.9%/<1%/2
ICCAD'22 [44]	ALL RTL bits	LR-MCP, Subset selection	Linear	✗	✗	Per-cycle	1×/1×	2.04%/87.2%	-/<0.1%/-
COBIT	ALL RTL bits	LR-MCP	**Boosting binary tree**	✔	✔	**Per-cycle**	**1×/1×**	**1.69%/95.5%**	**0.49%/<0.1%**/3

* The two values in each #Proxy and Performance cell are one-to-one matched. Note that 1× corresponds to 197 selected proxies.
† Each value pair in this column indicates the MAPE on the normal test set, along with the corresponding peak prediction success rate under a 5% error tolerance, based on our reproduction of the original inference model.
‡ The power, area overhead and latency (cycles) of an on-chip power meter.

hardware cost in its model design. Second, earlier works lack integrated, automated hyperparameter tuning frameworks, which limits their efficiency in developing power models for hardware-constrained deployment. For example, Lin et al. [32] adopt a simple grid search strategy to determine decision tree parameters, while Xie et al. [44], [45] do not provide an explicit strategy for tuning hyperparameters of weight shrinkage. Simmani [27] uses the Bayesian information criterion to guide feature selection but does not tune the regularization strengths in the elastic net, which implicitly influence the hardware implementation cost.

To address these challenges, we coordinate the binary traits of the bitwise proxy and the binary tree to construct our OPMs. Remarkably, the proposed OPM effectively processes proxy features in a lightweight inference backbone, leading to a low-overhead implementation while maintaining high accuracy, thus addressing the concern raised in 2). Furthermore, each tree node in the OPM detects the toggling of power proxies related to itself and correlates this information with its child nodes, fully leveraging the small set of proxy features to enhance accuracy, thereby overcoming the concern in 1). Additionally, COBIT formulates the model design as a multiobjective optimization problem and leverages dedicated optimization algorithms to automatically search for configurations that achieve an effective tradeoff between accuracy and hardware cost, thus meeting the requirements outlined in 3).

B. Power Peak Concern

Given the importance of power peaks in a processor, power composition is reviewed. Power consumption in digital integrated circuits is comprised of three major components: leakage power, short-circuit power, and switching power. The dynamic output switching power P_{sw} is dissipated through the charging and discharging of the total load capacitance at the device output pins. Therefore, it constitutes the primary code-related component of dynamic power, typically estimated based on toggle activity and load capacitance

$$P_{\text{sw}} = \alpha C_{\text{Load}} V^2 F. \tag{1}$$

For the cycle-accurate mode, it can be further simplified as the scaled sum of the effective capacitance of toggling pins [45]

$$P_{\text{sw}}[i] = \frac{1}{2} V^2 \sum_{p \in \{\text{ toggling pins }\}} C_{\text{effective}}(\text{pin}). \tag{2}$$

However, glitch power is also associated with toggle activities, arising from the inconsistency in the arrival time of multiple input pins of combinational logic devices. As a result, the proportion of glitch power is incorporated in dynamic power and strongly related to circuit activities, resulting in power peaks revealed in dynamic power analysis. If considering switching power only, it is reasonable to linearly model the effective load capacitance associated with pins. Nevertheless, linear models are unable to model the correlation between pins. Therefore, it fails to characterize the nonlinear power portion generated by the correlation, which is a significant part of dynamic power, especially in a compute-intensive design

$$P_{\text{glitch}}[i] = \sum_{i,j,..,k \in Q\text{proxies}} W(i, j, \ldots, k). \tag{3}$$

Taking this into consideration, COBIT utilizes a nonlinear regression model to enhance the learning of the nonlinear power contribution, improving the overall prediction accuracy.

As shown in Table I, none of the prior works has conducted a dataset-level evaluation specifically targeting power peak prediction. Previous methods report per-cycle or average-case metrics without assessing model performance under power peak datasets. In contrast, COBIT addresses this gap by explicitly evaluating its performance on predicting peak power, which is crucial for real-time power management and voltage regulation.

C. Integrated With Power Management

The low-latency prediction capability of COBIT enables it to function as a real-time telemetry unit for dynamic power management schemes, such as DVFS and AVS [17], [38]. Integration with existing power management frameworks can be achieved through multiple approaches. At the software level, COBIT's per-cycle power estimation can be accessed

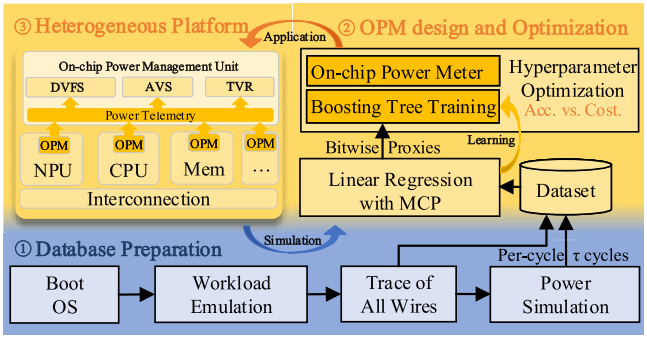

Fig. 2. Overview of COBIT framework.

by the hosted design to update control and status registers within the CPU core. These values can then be utilized by a runtime-level DVFS controller to adjust voltage and frequency settings, consistent with prior software-managed power scheduling frameworks [42].

Alternatively, for latency-sensitive power regulation scenarios, COBIT's output can be directly interfaced with hardware-level voltage control units to form a fast feedback loop without involving software intervention. This approach is aligned with prior studies that leverage machine learning-based power estimation to drive hardware-level power capping and adaptation in real time [23], [46].

III. COBIT Methodology

A. Overview

In this article, we propose the COBIT framework for OPM design and optimization as depicted in Fig. 2. COBIT aims to extract bitwise wires as power proxies from all RTL wires and build an accurate power prediction model along with a consistent OPM with low latency and small overhead. To accomplish the target, the framework can be segmented into three parts.

The first part shows the database preparation flow, which generates activity and power traces for heterogeneous designs on a heterogeneous platform using an emulator and a power analysis tool.

The second part outlines the methodology for our OPM design and optimization through a two-stage machine learning-based power modeling process. First, linear regression with MCP (LR-MCP) is employed to select the most representative bitwise wires as power proxies. Then, the toggling features of these proxies are utilized to train a boosting model of binary trees for power prediction. To balance prediction accuracy and hardware overhead, we propose a multiobjective optimization procedure for hyperparameter tuning of our boosting-tree-based predictor.

The third part illustrates the application of our OPMs for on-chip power management on heterogeneous platforms. The optimized OPMs are integrated into the components they model. These meters provide accurate cycle-by-cycle power consumption monitoring with low latency at runtime, assisting with various power management techniques.

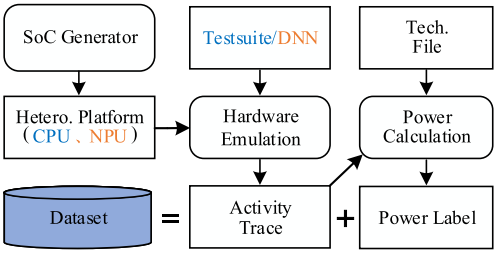

Fig. 3. Dataset preparation flow of heterogeneous hardware.

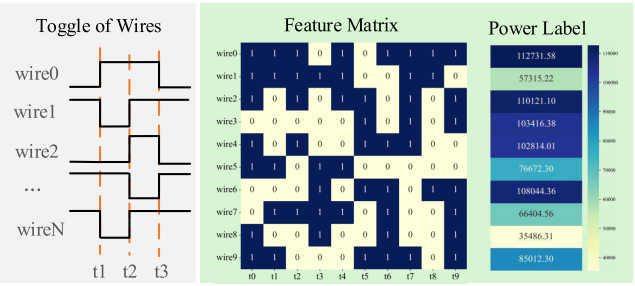

Fig. 4. Feature matrix and power labels in power datasets.

B. Dataset Preparation

As depicted in Fig. 3, the proposed flow integrates all steps of preparing the power datasets for heterogeneous designs. First, we establish the heterogeneous platform, which includes a superscalar out-of-order CPU core and an NPU. Next, the hardware emulation is used to execute different workloads–testsuite for the CPU and deep neural network inference for the NPU. The activity traces dumped from the emulator contain all the toggle information for all wires in a target design. Fig. 4 illustrates the method of extracting toggle information as the feature matrix and power labels as the ground truth. As discussed earlier, toggles on wires are the primary source of dynamic power consumption. Therefore, the toggle of each wire is extracted and modeled as a feature. A toggle (i.e., from 0 to 1 or from 1 to 0) is marked as 1, while nontoggles are marked as 0. Assuming that the target design has M bits, concatenating all features across N cycles results in a feature matrix $0,1^{N\times M}$. Many industrial power analysis tools support cycle-accurate power calculation at the RTL level, which generates time- and average-based power as the power label y_i. The feature matrix and power labels together constitute the dataset for modeling power patterns in heterogeneous platforms.

C. OPM Design and Optimization

We propose a two-stage power modeling method to capture the complex power patterns in heterogeneous designs and to identify the optimal power prediction model using multiobjective optimization algorithms. Then, we design the OPM for these models and integrate them with their respective target designs. Algorithm 1 outlines the key steps of our two-stage power modeling algorithm.

1) Proxy Selection: Proxy selection is a crucial step in reducing the feature space of a design with a large number

Algorithm 1 Two-Stage Power Modeling Algorithm for OPM Design and Optimization

Input: $\mathcal{D}, \boldsymbol{\lambda}, \gamma, CP, \boldsymbol{R\_rgs}, \boldsymbol{hp\_rgs}, T\_th$
Output:
Initialize $\boldsymbol{Q}, \boldsymbol{\mathcal{D}\_proxy}, \boldsymbol{BT}$
for each λ in $\boldsymbol{\lambda}$ **do**
  // stage 1
  $proxy\_idx, Q$ = LR-MCP($\mathcal{D}, \lambda, \gamma$)
  $\mathcal{D}\_proxy = \mathcal{D}[proxy\_idx]$
  $\boldsymbol{Q}$.append(Q)
  $\boldsymbol{\mathcal{D}\_proxy}$.append($\mathcal{D}\_proxy$)

  // stage 2
  Initialize $\boldsymbol{POF} = []$
  for each R in $\boldsymbol{R\_rgs}$ **do**
    POF_R = CP($\mathcal{D}\_proxy$, R, $\boldsymbol{hp\_rgs}$)
    $\boldsymbol{POF}$.append(POF_R)
  end for
  BT = NonDominated($\boldsymbol{POF}$).find(T_th)
  $\boldsymbol{BT}$.append(BT)
end for
return $\boldsymbol{BT}$

of M wires to a manageable set of power proxies. Previous methods such as Primal [48], Simmani [27], APOLLO [45], and DEEP [44] have used techniques like principal component analysis, K-means++, and LR-MCP to identify power proxies. Among these, LR-MCP efficiently and effectively selects features from all signals/wires in a circuit, which can then be used to train the subsequent regression model. Therefore, we adopt LR-MCP to filter bitwise wires as power proxies.

Stage 1 of Algorithm 1 describes the power proxy selection procedure. Given the dataset $\mathcal{D}$, LR-MCP filters Q proxies from the M wires. The penalty term for the weights in the linear model, the MCP, controls the range and rate of weight decay. Its derivative is formulated as follows:

$$\left|\frac{\partial P_{\mathrm{MCP}}\left(w_j', \gamma > 1\right)}{\partial w_j'}\right| = \begin{cases} \lambda - \dfrac{|w_j'|}{\gamma}, & \text{if} |w_j'| \le \gamma\lambda \\ 0, & \text{if} |w_j'| > \gamma\lambda \end{cases} \tag{4}$$

where γ and λ are the hyperparameters of the MCP term that can be adjusted to influence the number of power proxies Q.

2) Lightweight Binary-Tree-Based Regressor: After selecting the power proxies, COBIT employs boosting learning to construct an accurate prediction model. For a given dataset $\mathcal{D}\_proxy = \{(\mathbf{x}_i, y_i)\}$ ($|\mathcal{D}| = N, \mathbf{x}_i \in \{0,1\}^Q, y_i \in \mathbb{R}$), the boosting model consists of R trees to predict the power consumption. In the rth boosting round, a new binary tree is constructed to fit the residual between the ground truth and the estimated values from the previous $r-1$ trees. The loss function of boost learning in the rth round can be formulated as $\mathcal{L}$ [15]

$$\mathcal{L}^{(r)} = \sum_{i=1}^{N} l\left(y_i, \hat{y}_i^{(r-1)} + f_r(\mathbf{x}_i)\right) + \sum_{k=1}^{r} \Omega(f_k)$$
$$\text{where } \Omega(f_k) = \gamma' T + \alpha \sum_{j=1}^{T} |w_j| + \frac{1}{2}\lambda' \sum_{j=1}^{T} w_j^2 \tag{5}$$

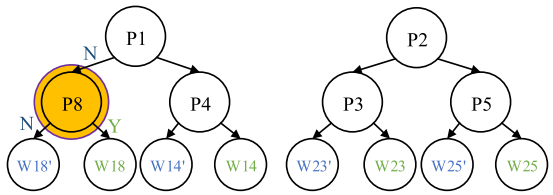

Fig. 5. Example of boosting model with two binary trees.

where l is a differentiable convex loss function, typically squared error for regression tasks, and $f(\cdot)$ is the inference function of a tree. The regularization term $\Omega(f)$ penalizes the number of leaf nodes T and weights w_j in a tree. After performing a second-order Taylor expansion and removing constant, the optimal value of the weight and $\mathcal{L}$ can be calculated as

$$w_j^* = -\frac{G_j}{H_j + \lambda'}, \qquad \hat{\mathcal{L}}^{(r)} = -\sum_{j=1}^{T} \frac{G_j^2}{2\left(H_j + \lambda'\right)} + \gamma' T \tag{6}$$

where G_j and H_j are the sum of first- and second-order gradient statistics on the loss function, respectively. Thus, the rule of splitting a tree node is to evaluate the loss reduction resulting from the split as

$$\mathcal{L}_{\text{split}} = \frac{1}{2}\left(\frac{G_L^2}{H_L + \lambda'} + \frac{G_R^2}{H_R + \lambda'} - \frac{\left(G_L^2 + G_R^2\right)}{H_L + H_R + \lambda'}\right) - \gamma'. \tag{7}$$

Using a greedy growth policy, the tree continues to split a node as long as the gain from the split remains significant. The splitting stops when the gain falls below a predefined threshold or the maximum depth constraint is reached. Once a node ceases to split, it becomes a leaf node, and the corresponding leaf score, w, is determined. These leaf scores not only represent the weights in our power model but also contribute to the main overhead in the OPM implementation. Additionally, the splits are constrained by the *max_leaves* parameter, which limits the number of leaves in a single tree. Finally, after R boosting rounds, the boosting model aggregates the leaf scores from all R trees to predict the power consumption.

Discussion of Our Model: COBIT takes advantage of the binary nature of wire toggles, which aligns perfectly with the growth characteristics of a binary tree. Each bitwise proxy, with its two possible states, can be effectively modeled as splits in a nonleaf node of a binary tree, where toggle and nontoggle states represent the two split directions. This coordination between binary inputs and binary branches offers two key benefits. First, binary trees efficiently process input features in a highly structured manner, ensuring optimal feature utilization. Second, the binary nature of the inputs simplifies the decision-making process for node inference, significantly reducing the overhead associated with model structure implementation.

Compared to linear models, the nonlinear boosting tree model offers two significant advantages, both during model training and inference, enabling the capture and modeling of the nonlinear part of dynamic power.

First, during tree construction, the correlation between bitwise proxies is considered in each tree. As shown in Fig. 5,

TABLE II
HYPERPARAMETERS OF BOOSTING TREE MODEL CONSIDERED IN THE HPO PROCEDURE

Hyperparameter	Description	Range
max_leaves	Maximum number of nodes to be added	[16, 64]
max_depth	Maximum depth of a tree	[4, 32]
α	L1 regularization term on weights	[1e-8, 1.0]
λ'	L2 regularization term on weights	[1e-8, 1.0]
γ'	Minimum loss reduction required to further split a leaf node	[1e-8, 1.0]
min_child_weight	Minimum sum of instance weight (hessian) needed in a child	[1e-8, 5.0]
$subsample$	Subsample ratio of the training instances	[0.6, 1.0]
$colsample\_bytree$	The subsample ratio of columns when constructing each tree	[0.6, 1.0]
η	Step size shrinkage used in update to prevents overfitting	[1e-8, 1.0]

if Proxy 1 (P1) does not toggle, the tree will then evaluate whether Proxy 8 (P8) toggles. Depending on this evaluation, the tree will follow either the left or right path to the corresponding leaf node. Both leaf nodes contain nonzero weights, demonstrating that the tree model can learn the correlation between proxies and capture the nonlinear power behavior. This ability is crucial for building an accurate power model, especially when addressing power peaks in compute-intensive designs such as NVDLA.

Second, during model inference, each binary tree determines the path to a leaf node based on the toggle state of the bound proxy. Thanks to the simplicity of the proxy wire toggles and the lightweight structure, our boosting tree model can quickly predict power consumption, making it as fast as a linear model in terms of inference speed. Additionally, the decision path traverses a single chain of tree nodes, resulting in low dynamic power overhead in the OPM implementation. Furthermore, since the inferences of all trees can be executed in parallel, the inference latency is independent of the number of trees (R), providing a significant advantage for latency optimization.

For the multicycle prediction task over a time interval, COBIT estimates the power consumption by averaging the predictions of the per-cycle/multicycle$^\tau$ models across t cycles. Typically, τ, t are selected to be powers of 2, optimizing the design for efficient hardware implementation. Due to the advantages of the proposed model, the multicycle model also demonstrates superior prediction accuracy.

With the proposed analytical power model, the design-time power analysis flow can be greatly simplified. By using the traces of only a few hundred bitwise wires as proxy features, our power model enables fast and accurate per-cycle and multicycle power predictions across a broader range of workloads.

3) Design Space Exploration of Hyperparameters: The per-cycle power model is trained using boosting learning, which involves numerous tunable hyperparameters. Table II outlines the considered hyperparameters and their descriptions. Specifically, α and λ' are the coefficients for regularization terms on the leaf weights in trees, while γ' and min_child_weight influence the splitting of tree nodes. The parameters $subsample$ and $colsample\_bytree$ help mitigate the risk of overfitting and promote diversity within the boosting model, respectively. Additionally, η improves the model's generalization capability by controlling the step size. Consequently, these hyperparameters play a significant role in determining both prediction accuracy and model robustness. Furthermore, the overhead associated with the OPM implementation is preferably considered during model construction.

To address these two aspects, we propose a multiobjective HPO toolbox that automatically identifies the optimal model solution. The multiobjective optimization problem in COBIT is a minimization problem, which can be formulated as

$$\begin{aligned} &\text{Min. } F(\mathbf{x}) = \left(\text{Error}\left(\mathbf{W}_{z(\mathbf{x})}\right), \quad \text{Cost}\left(\mathbf{W}_{z(\mathbf{x})}\right)\right) \\ &\text{s.t. } \mathbf{x} \in \mathcal{D}. \end{aligned} \tag{8}$$

The proposed model is formulated as a mapping function $z(\mathbf{x})$ that determines the leaf score based on the feature vector $\mathbf{x}$. All leaf scores in binary trees collectively form a weight matrix $\mathbf{W}$. In our multiobjective optimization problem, each parameter is sampled within the specified range. Among these, $\eta, \gamma', max\_leaves$, and min_child_weight are considered on a logarithmic scale. The first objective of the HPO is prediction accuracy.

As the storage and access requirements for the leaf weights are the primary sources of overhead in our OPM implementation, the number of leaf nodes in all trees is selected as the second optimization objective. This ensures that the overhead is considered during the model training process.

The various hyperparameter choices define the design space of our power model. To effectively explore this complex design space and identify the Pareto optimal front in the objective space, we leverage three powerful multiobjective optimization algorithms—NSGA-II [18], NSGA-III [25], and Bayesian optimization (BO) algorithm [43]. For comparison, we also investigate the random sampler. To accelerate the search, we employ pruners that automatically terminate unpromising trials during the HPO process. These include Hyperband [30] and median pruners [11].

Overall, we investigate the eight pairs of samplers and pruners using Algorithm 2 to explore the design space of the proposed power model. Note that $\mathcal{D}\_proxy$ used in this process is generated in stage 1 of Algorithm 1. Between lines 12 and 24 of Algorithm 2, all sampler–pruner pairs are traversed under different population size settings. The POF is computed during each HPO process, and evaluation metrics are employed to compare the performance of these pairs. Finally, we select the candidate pair (CP) from all pairs to search for the optimal power model for metric comparison with existing methods.

To meet different implementation requirements, the number of power proxies and the model size can vary. With the CP,

Algorithm 2 Multiobjective Design Space Exploration of the Boosting Model Based on Binary Trees

Input: $\boldsymbol{S}, \boldsymbol{P}, \boldsymbol{hp\_rgs}, \mathcal{D}\_proxy$
Output: CP

1: global N
2: **function** HyperParameterOptimize(s, p, n, $\boldsymbol{hp\_rgs}$)
3: **for** iteration i **do**
4: use sampler s to search n trials with different hyperparameter combinations.
5: train n models with n set of sampled hyperparameter combinations.
6: validates the accuracy and leaf number of the models.
7: save the hypervolume convergency history.
8: **end for**
9: Pareto trials = NonDominated(N trials)
10: **return** Pareto trials.
11:
12: Initialize population size list $l = [16, 32, 48]$,
13: Region of interest $roi = [5\%, 800/1000]$.
14: **for** each n in l **do**
15: $\boldsymbol{pair\_metrics} = []$
16: **for** each s in $\boldsymbol{S}$ **do**
17: **for** each p in $\boldsymbol{P}$ **do**
18: Pareto optimal front POF = HPO(s, p, n, $\boldsymbol{hp\_rgs}$)
19: Compute the hypervolume hv, the spacing $spacing$, and the number of Pareto trials of the POF.
20: $\boldsymbol{pair\_metrics}$.append($\{s, p, hv, spacing\}$)
21: **end for**
22: Compute the coverage matrix **Cov** and net coverage $\boldsymbol{NetCov}$ of the POFs.
23: **end for**
24: **end for**
25: find CP with best performance.
26: **return** CP

stage 2 in Algorithm 1 outlines the key steps for searching and training our model. In the outer loop, bit-level LR-MCP iterates over all possible values of λ to generate different numbers of proxies Q, along with their corresponding proxy datasets $\mathcal{D}\_proxy$. The inner loop (lines 11–14) explores a wide range of boosting rounds R, representing varying model size and regression capabilities. Line 12 uses the CP to explore the design space of the power model with a specific set of Q and R, where N hyperparameter combinations (trials) are searched. The search process outputs a Pareto optimal set POF$_R$. After completing the inner loop, the model at a specific Q has its overall Pareto set **POF**. Line 15 employs the *NonDominated* method to filter the final Pareto set in **POF** and outputs the best trial (BT) closest to the T_th constraint.

With the proposed HPO framework, multiple power models with various Q proxies can be developed, each offering a balanced tradeoff between accuracy and overhead for OPM implementation. Consequently, COBIT is capable of providing diverse OPM solutions, eliminating the need for manual design and tuning.

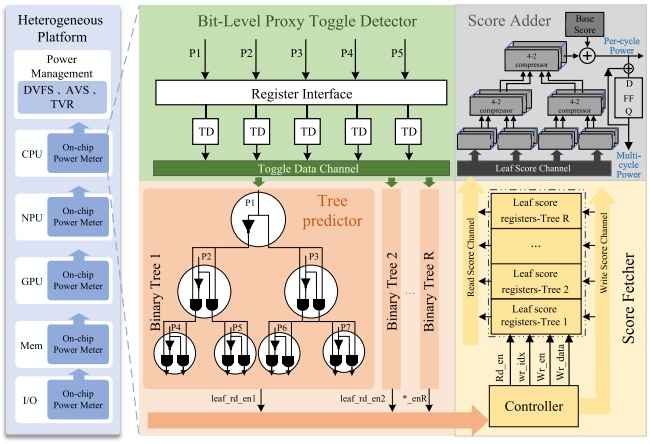

Fig. 6. OPM architecture of the boosting model based on binary trees.

4) OPM Hardware Design: After obtaining the optimal power model with a specific set of power proxies, we design the architecture of the OPM of the model, as shown in Fig. 6.

The toggle detectors first use registers to store the voltage levels of all bitwise proxies. Then, at each cycle, toggles are extracted as features using XOR gates. These features are passed to the boosting tree predictor, which consists of R hardware-implemented trees. Each tree node is composed of two AND gates and an inverter for node inference. The AND gate at the current node combines the output of its parent node with the toggle of the proxy it detects, capturing the parent–child correlation. The inverter is used to implement the two split directions of a nonleaf node.

During the inference of a single tree, only one leaf node is reached along the decision path, and the *leaf_read_en* signal is enabled and output to the score controller through the *read control channel*. The score fetcher then reads the corresponding entries from the score registers in parallel for all R trees via the *read score channel*. Finally, the score adder sums the indexed fixed-point weights using a Wallace tree adder.

Despite being a nonlinear predictor, the boosting-tree OPM achieves low latency due to its optimized algorithm and hardware design. The binary growth characteristic of the tree model aligns seamlessly with the binary nature of bitwise proxies, enabling efficient processing of proxy inputs compared to other nonlinear algorithms. Furthermore, the tree structure is implemented using only combinational gates, minimizing delay. As a result, the overall computation latency of the OPM is primarily determined by the time required to read and sum the leaf scores.

It is important to highlight that the OPM controller can rewrite leaf scores via the *write score channel*. This programmability allows the weights to be calibrated based on hardware measurements of power consumption on silicon, accommodating variability in the manufacturing process and enabling adaptability for unforeseen applications. Furthermore, the RTL design and integration of the OPM can be automatically configured and generated through scripts, significantly enhancing the feasibility and portability for diverse

heterogeneous components. As shown in Fig. 2, the OPM derived from our model can be fully integrated into heterogeneous designs, such as CPU and NPU.

IV. Experimental Results

A. Setup

We utilize XGBoost 2.0.1 [15] to construct our power model and skglm 0.3 [13] to reproduce APOLLO [45] and DEEP [44] models, while Simmani [27] has made its source code publicly available. Scikit-learn [37] 0.18.1 is used to reproduce the results of the decision tree and the bagging tree method in [32] and [33]. For accuracy comparison, the mean absolute percentage error (MAPE) and the coefficient of determination (R^2) are used to evaluate the COBIT and other works. All models are developed using Python 3.8

$$\text{MAPE} = \frac{1}{N}\sum_{i=1}^{N}\frac{|y_i - \hat{y}_i|}{y_i}, \text{R}^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

1) Dataset Preparation: To model and learn the complex power patterns in heterogeneous platforms, we establish a heterogeneous platform with the dual-issue BOOM [47] CPU core and the small NVDLA [1] using chipyard [12]. BOOM is a representative out-of-order processor core that competes performance-wise with commercially high-performance cores deployed in datacenters [47]. We run the riscv tests [8] suite for the training set and the CoreMark [5] benchmark suite for the test set. The riscv tests suite includes eight microbenchmarks: dhrystone, mm, multiply, median, vvadd, towers, spmv, and qsort. CoreMark, a well-established benchmark, provides both practical insights and comprehensive coverage of CPU operations. NVDLA [1] is an open-source architecture designed to standardize the creation of deep learning inference accelerators. For NVDLA, we execute three DNN inference tasks—Lenet, ResNet-50, and ResNet-18—and select multiple trace windows to capture the rich power patterns of NVDLA. It is worth noting that the number of RTL bits in NVDLA and BOOM is 701914 and 689493, respectively. Palladium Z1 [7] is used to perform emulation and collect traces, offering ultrafast in-circuit emulation with cycle-accurate resolution. PowerPro [2] is employed for cycle-accurate and average power consumption evaluation under the TSMC 28-nm CMOS process. The sizes of the per-cycle training and test sets for BOOM and NVDLA are 34717/15000 and 31021/15000, respectively. The test sets are labeled as Normal-Power datasets, which retain power data for continuous cycles without specifically distinguishing power variations. To address the absence of power peak concerns in existing works, we construct a Peak-Power test set for NVDLA to evaluate the prediction accuracy of various methods on power peaks. This test set is generated using the sliding window method: the continuous power trace is partitioned into smaller windows, and for each window, the mean and standard deviation σ are computed. Peaks are identified as cycles where the power consumption exceeds 3σ from the mean. Specifically, the 4269 peak samples were selected from power traces of three neural network workloads of different sizes: LeNet (small), ResNet-18 (medium), and ResNet-50 (large). Together, these traces exceed 517000 cycles of dynamic power data. These workloads vary in size and computational complexity, ensuring that the Peak-Power dataset captures different load intensities and execution behaviors. As a result, the Peak-Power test set exhibits significantly higher power amplitudes compared to the continuous Normal-Power test set and provides a representative and diverse basis for evaluating peak prediction performance. Each sample in our dataset for the per-cycle model corresponds to a single clock cycle, capturing the toggle state of proxies and its associated power label. This provides precise per-cycle granularity for training and evaluation.

2) HPO Setting: Optuna 3.5.0 [11] provides various samplers (NSGA-II, NSGA-III, BO, and Random) and pruners (Hyperband and Median). For the BO sampler, the tree-structured Parzen estimator (TPE) [36] is used as the surrogate model, while the expected hypervolume improvement (EHVI) serves as the acquisition function. We evaluate these sampler–pruner pairs using five metrics. The hypervolume (HV) indicator is a widely used set-quality metric for evaluating the Pareto front. Coverage is another metric that compares and evaluates the dominance between two Pareto fronts, measuring how well one Pareto front dominates or covers the other. Additionally, we define Net Coverage, which represents the dominance advantage over the disadvantage of being dominated between one Pareto front i and others j

$$\text{Cov}(i,j) = \frac{|\{b \in j \mid \exists a \in i : a \text{dominates } b\}|}{|j|}$$
$$\text{NetCov}[i] = \sum_j \text{Cov}(i,j) - \sum_j \text{Cov}(j,i)$$

where a and b are Pareto trials on i and j, respectively. The spacing indicator [19] is a quantitative metric of the standard deviation of the distance in objective space between Pareto solutions in the target space. The smaller it is, the more uniform the distribution of the Pareto set. We use pymoo 0.6.1 [14] to calculate this metric. Furthermore, the number of Pareto trials and the running time of HPO execution are collected

$$\text{Spacing} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(d_i - \bar{d})^2}.$$

To determine the candidate sampler–pruner pair for performance comparison, the boosting tree model is configured with 197 power proxies as input features and 50 boosting rounds.

B. Exploration of HPO

Fig. 7(a) shows all the Pareto trials generated by the eight pairs at a population size of 32. The MAPE of the models trained with these Pareto trials ranges from 1.42% to 31.9%, while the number of leaves varies from 670 to 3200. Table III summarizes the metrics of the overall Pareto trials for each sampler–pruner pair. Among all pairs, NSGAII+Hyperband (N2+H) demonstrates superiority in Net Coverage and Spacing metrics, achieving a NetCov of 3.31/3.98 and a Spacing of 33.84/38.61 at pop_size of 32/48. In contrast, the NetCov of the BO pairs is negative, indicating weak dominance compared

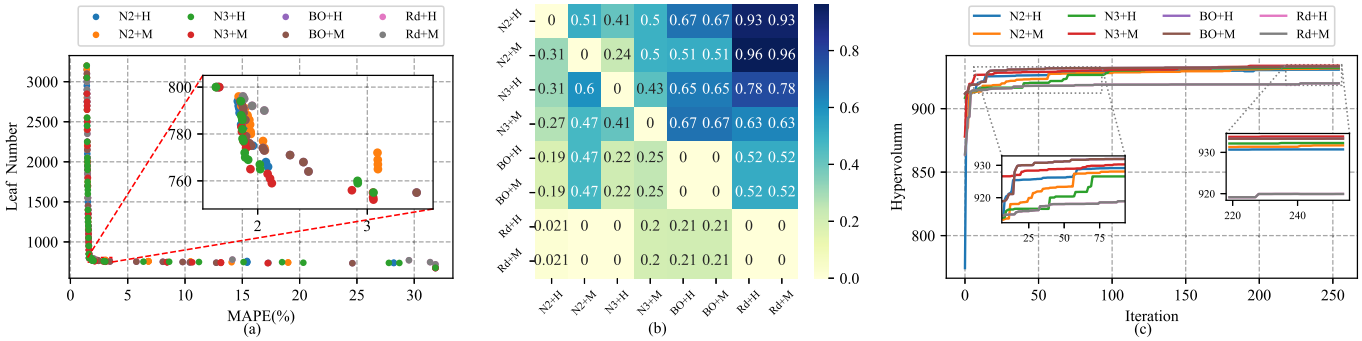

Fig. 7. (a) Distribution of Pareto-optimal front, (b) coverage matrix, and (c) HV convergence of eight sampler-pruner pairs with a population size of 32 in the HPO of NVDLA.

TABLE III
COMPARISON OF DIFFERENT MULTIOBJECTIVE OPTIMIZATION SAMPLER–PRUNER PAIRS

Pairs	Abbr.	Trials(#)	Runtime(hours) ↓	Pareto trials(#) ↑	Hypervolume ↑	Net Coverage ↑	Spacing ↓
Random + Hyperband	Rd+H	8192	10.45	27	919.92	-3.47/-3.69/-3.83	54.2
Random + Median	Rd+M	8192	37.38	27	919.92	-3.47/-3.69/-3.83	54.2
BO + Hyperband	BO+H	8192	42.26/40.9/31.85	46/43/42	931.64/933.34/932.07	-1.34/-0.77/-1.58	47.43/42.73/59.91
BO + Median	BO+M	8192	41.99/41.13/46.24	46/43/42	931.64/933.34/932.07	-1.34/-0.77/-1.58	47.43/42.73/59.91
NSGAII + Hyperband	N2+H	8192	9.51/10.93/9.12	52/48/50	932.19/930.73/932.28	2.73/**3.31/3.98**	35.25/**33.84/38.61**
NSGAII + Median	N2+M	8192	9.27/9.47/9.68	63/55/50	933.06/931.7/931.61	**3.14**/1.47/1.56	**27.91**/38.08/38.65
NSGAIII + Hyperband	N3+H	8192	7.99/10.5/7.68	52/46/53	932.34/932.28/932.88	2.9/2.7/2.41	34.86/47.66/54.63
NSGAIII + Median	N3+M	8192	7.64/7.42/6.8	51/44/40	**933.19/933.85/934.39**	0.87/1.42/2.87	36.65/43.62/44.98

to the evolutionary algorithms. Furthermore, the Spacing of N2+H is 33.84/38.61 at pop_size of 32/48, suggesting that the Pareto front produced by this pair is more evenly distributed in the objective space.

Fig. 7(b) presents the coverage matrix of the Pareto fronts generated by these pairs. Each value in the grid represents the coverage of the pair in a row over the pair in a column. The columns for Rd+H and Rd+M exhibit higher coverage values, indicating that the evolutionary and Bayesian samplers produce higher quality Pareto fronts than the Random samplers. Additionally, because the step of maximizing the acquisition function in BO ensures the search of promising trials, both BO+H and BO+M show identical Cov metrics. This suggests that the BO pairs are effective in searching for optimal trials. Fig. 7(c) illustrates the convergence of HV for all pairs running 8192 trials. The BO pairs demonstrate a higher HV than the other pairs at fewer than 100 iterations, clearly indicating their superior search effectiveness. However, the process is completed in 42.26/40.93/31.85 h for pop_size of 16/32/48, due to the inclusion of the surrogate model training and the computation of EHVI. In contrast, model-free evolutionary algorithms generate offspring trials based on simpler probabilistic mechanisms, resulting in faster execution.

Considering feasible solutions within the region of interest *roi*, where the OPM is preferably implemented, the region of [5%, 1000] is selected to extract Pareto trials from each pair. The performance in *roi* is shown in Table IV. The NSGAIII+Hyperband pair achieves a NetCov of 5.26/4.64/2.29 at pop_size of 16/32/48, demonstrating a dominance advantage over the other pairs. Furthermore, it has the highest HV of 8.09/8.08/7.99 in *roi*. In terms of overall runtime, this pair efficiently explores 8192 hyperparameter

TABLE IV
PERFORMANCE COMPARISON OF DIFFERENT SAMPLER–PRUNER PAIRS IN THE FEASIBLE REGION

Pairs	Pareto trials(#)	Net Coverage ↑	Hypervloume ↑
Rd+H	7	-4.56/-4.8/-6.0	7.05
Rd+M	7	-4.56/-4.8/-6.0	7.05
BO+H	18/18/15	-0.35/-0.57/0.29	7.88/7.96/7.99
BO+M	18/18/15	-0.35/-0.57/0.29	7.88/7.96/7.99
N2+H	12/13/13	2.95/3.21/**3.69**	8.06/7.83/7.98
N2+M	14/20/15	-0.51/-0.75/-0.75	7.92/7.76/7.78
N3+H	17/18/17	**5.26/4.64**/2.29	**8.09/8.08/7.99**
N3+M	17/16/17	2.11/3.62/6.21	8.1/8.15/8.28

configurations, with an average runtime of 8.72 h. Consequently, we designate NSGAIII+Hyperband as the selected HPO algorithm pair (CP) for the subsequent performance comparison with existing methods.

Overall, with the candidate searcher and pruner, the HPO framework in COBIT significantly reduces the modeling effort involved in manually designing and tuning OPMs, achieving an optimal tradeoff between accuracy and overhead.

C. Accuracy on NVDLA

1) Per Cycle: In order to build the per-cycle predictor for NVDLA, we conduct matrix-like HPOs using the CP at stage 2 in Algorithm 1 with different Q inputs across a wide range of R. Then, the Pareto optimal fronts (POFs) for all Q can be derived. Consequently, the BT at each Q is selected from the POF by setting the T_th threshold at 800. We extract the hyperparameters from the BT for each Q and proceed with retraining and testing. The training process for our models is notably fast. For instance, training boosting tree models with

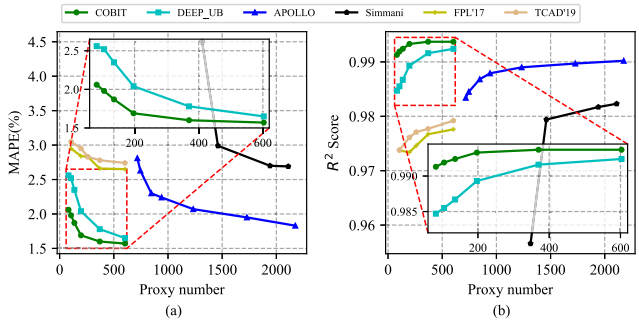

Fig. 8. Per-cycle (a) MAPE and (b) R^2 score versus the number of bitwise proxies for power prediction on NVDLA under Normal-Power datasets.

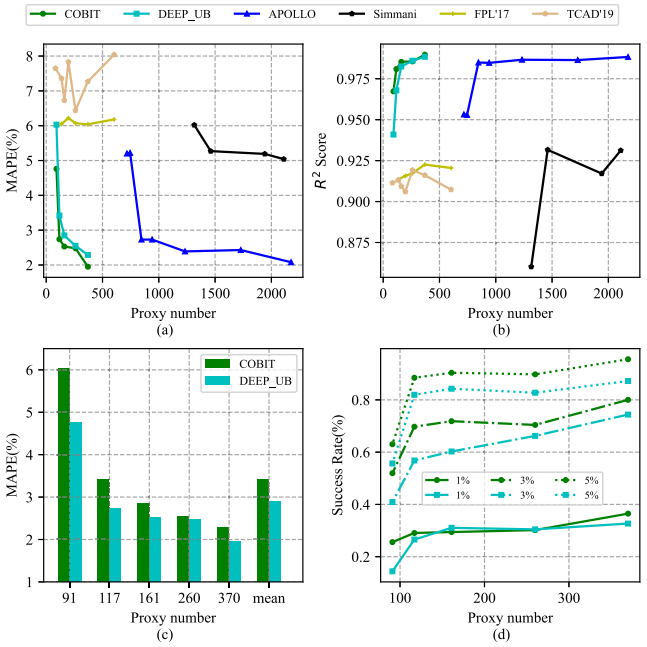

Fig. 9. For power prediction on NVDLA under Peak-Power dataset. (a) Per-cycle MAPE and (b) per-cycle R^2 score versus the number of bitwise proxies, (c) MAPE comparison with DEEP_UB, and (d) success rate comparison with DEEP_UB under different APET settings.

365 and 250 proxies takes 98.04 and 42.91 s, respectively, for 50 boosting rounds, utilizing 32 parallel threads on a 64-core Intel Xeon Gold 5218 server at 2.3 GHz.

For the per-cycle prediction task, DEEP [44] and APOLLO [45] are recognized as the comparison works for linear predictors, while Simmani serves as a representative work for the nonlinear predictors. The decision tree and the bagging tree used in [32] and [33] are compared with ours because of a similar model structure. From the perspective of input feature, APOLLO and Simmani extract signal-level proxy, with the number of bits contained in the selected signals used as the abscissa for comparison with our method. We reserve all candidate proxies in our DEEP reproduction, thereby comparing with a theoretical upper bound of DEEP's accuracy labeled as DEEP_UB.

Fig. 8 clearly demonstrates the superior performance of COBIT in per-cycle power prediction on the NVDLA. With an MAPE of only 1.69% at $Q = 197$, COBIT achieves an exceptional prediction accuracy while using just 0.028% of the total RTL bits in NVDLA—substantially fewer resources compared to many methods. In contrast, the DEEP_UB method, despite keeping all proxies without subset selection, struggles to achieve an MAPE of 2.04%, showcasing COBIT's clear advantage in accuracy. When compared to traditional decision tree and bagging tree methods, they reach the MAPE of 2.84% and 2.95%, respectively, 1.68× and 1.75× higher than ours. Moreover, when comparing to signal-level methods like APOLLO, which requires 100 signal-level features with a total of 941 bits (4.78× more than COBIT), the MAPE of APOLLO stands at 2.24%, 1.33× higher than COBIT's. Simmani, another prominent nonlinear method, achieves the MAPE of 4.5% with 1315 proxies, indicating even poorer performance with much more proxies compared to ours.

In summary, COBIT surpasses both bit-level methods [44] (linear), [32], [33] (nonlinear), and signal-level methods [45] (linear), [27] (nonlinear) in terms of prediction accuracy. The advantage is more apparent when using fewer input features. This not only demonstrates its superior efficiency but also highlights COBIT as a more scalable solution for per-cycle power prediction.

On the Peak-Power test set, COBIT demonstrates a significant accuracy advantage, further showcasing its capabilities, as illustrated in Fig. 9(a) and (b). Leveraging the fitting ability of the nonlinear boosting model, COBIT achieves an MAPE of 1.94% with just 370 bitwise proxies. In contrast, APOLLO requires 452 proxies to reach an MAPE of 5.21%. This means that COBIT utilizes approximately 82% of the proxies required by APOLLO while achieving 2.69× lower on prediction error. Fig. 9(c) and (d) further highlights the performance comparison between DEEP_UB and COBIT. As shown in Fig. 9(c), COBIT achieves an MAPE that is 0.79×/0.8× smaller than the linear model in DEEP_UB at $Q = 91/117$. This demonstrates the advantage of COBIT's nonlinear regression, requiring fewer proxies while outperforming the linear model in terms of prediction accuracy. Fig. 9(d) shows the success rate of predictions with errors falling below different absolute percentage error thresholds (APETs). Under the 5% APET setting, COBIT achieves a success rate of over 95.5% at $Q = 370$, while DEEP_UB struggles with only 87.2%. For smaller Q, COBIT also attains a success rate of 69.7% at the 3% APET setting, surpassing DEEP_UB's 56.8% at $Q = 117$. Overall, COBIT achieves an average improvement of 3.22%/8.11%/6.12% and a max improvement of 5.36%/12.93%/8.34% over DEEP_UB at 1%/3%/5% APET. These results highlight that the nonlinear boosting model in COBIT significantly outperforms the linear model in DEEP_UB in terms of predicting power peaks on NVDLA.

The accuracy of predicting power peaks with our per-cycle model is further demonstrated by comparing the power labels and the power prediction trace of our model on the 12000-cycle window, as shown in Fig. 10. Power peaks are characterized by significant changes in power consumption, which correspond to sudden current variations due to hardware activity. These rapid changes can lead to an increase in $L \times dI/dt$, causing potential voltage droop. As seen in the two subplots, the power prediction from our model closely matches

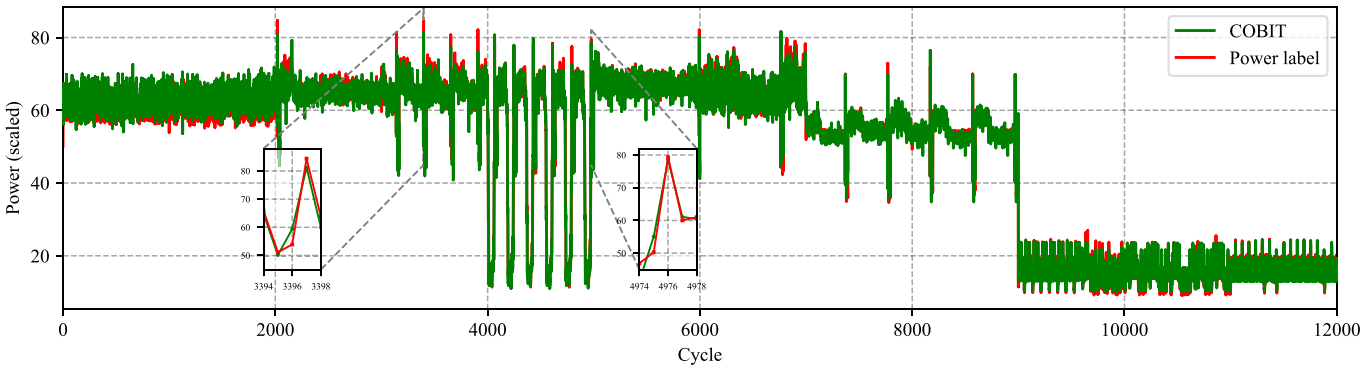

Fig. 10. Power prediction trace of a COBIT per-cycle model with $Q = 370$ and $R = 50$ on small NVDLA.

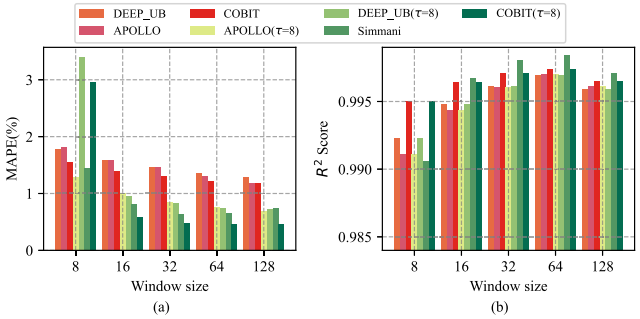

Fig. 11. Multicycle (a) MAPE and (b) R^2 score versus window size for power prediction on NVDLA.

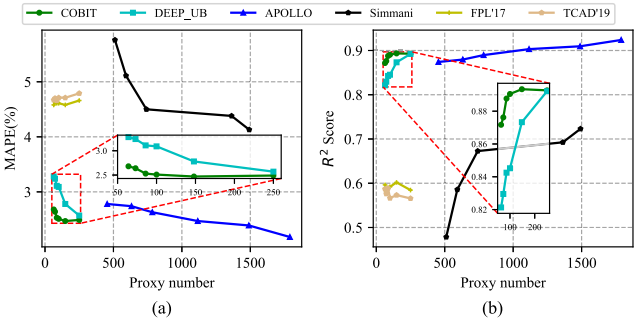

Fig. 12. Per-cycle (a) MAPE and (b) R^2 score versus the number of bitwise proxies for power prediction on BOOM CPU core.

the power label. This indicates that our model effectively tracks current transients, enabling proactive activation of circuit-level mitigation techniques, such as TVR, before voltage droop becomes problematic.

2) Multicycle: For the comparison of multicycle prediction, Fig. 11 illustrates the multicycle performance of all models over measurement windows of $t \in \{8, 16, 32, 64, 128\}$. Notably, the multicycle power label for each window size t is measured by PowerPro using a trace of 49000 cycles. For APOLLO, DEEP_UB, and COBIT, we implement the multicycle predictions across five window sizes based on their respective per-cycle models and multicycle models.

In Fig. 11(a), the first three columns for each window size represent the MAPE of multicycle prediction using the *per-cycle* model of each approach. The prediction for each window size is obtained by simply averaging the predictions of the *per-cycle* model over t cycles. In contrast, the last four columns, except for Simmani$^{\tau}$, represent the MAPE of the corresponding multicycle ($\tau = 8$) model. For these, the prediction on t cycles is averaged from the predictions of each multicycle model. Specifically, we also conduct HPO for the COBIT multicycle predictor. Five Simmani$^{\tau=t}$ multicycle models are trained, where the number of clusters K in Simmani is determined within the range of [50, 300], generating signal-level proxies of sizes $[243, 238, 296, 297, 295]$.

In this context, we compare the APOLLO *per-cycle* model ($Q = 941$) and APOLLO$^{\tau=8}$ *multicycle* model ($Q = 633$) with the DEEP_UB/COBIT *per-cycle* model ($Q = 197$) and *multicycle* ($\tau = 8$) model ($Q = 147$). As shown in Fig. 11(a), the COBIT *per-cycle* model demonstrates superior accuracy compared to both DEEP_UB and APOLLO across all measurement windows, using only 21% of the proxies required by APOLLO. Additionally, COBIT$^{\tau=8}$ outperforms APOLLO$^{\tau=8}$, DEEP_UB$^{\tau=8}$ and Simmani$^{\tau}$ when measurement window $t \geq 16$. Specifically, at $t = 32$, it achieves $MAPE = 0.48\%, R^2 = 0.99$ using just 23% proxies required by APOLLO$^{\tau=8}$. These results reinforce the discussion in Section III-C1, showcasing that both the per-cycle and multicycle models in COBIT are highly effective at providing accurate and efficient power predictions.

D. Accuracy on BOOM

For the BOOM CPU core, we perform HPOs with varying Q in the range of $R \in [20, 90]$ and then combine the POFs of all R to a united Pareto front for a specific Q, as shown in line 13 in Algorithm 1. From this unified Pareto front, the BT can be selected in line 15 by restricting the threshold T_th to 1000. After retraining with these optimal hyperparameters, COBIT also demonstrates its capability for the CPU core.

Fig. 12 shows the prediction accuracy of the per-cycle model on BOOM, achieving the MAPE of 2.68%/2.49% with 64/250 bitwise power proxies. Compared with baselines, the accuracy of our power model leads at $Q \leq 250$, which indicates the capability of our nonlinear predictor in the case of fewer inputs, thus addressing the concern 1) raised in Section II-A. Additionally, our multicycle model on BOOM also exhibits a high accuracy of 2.49% MAPE at $t = 128$ window. Using

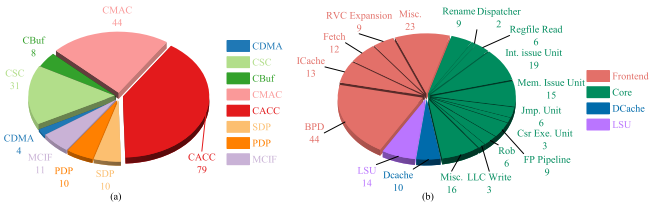

Fig. 13. Distribution of the selected power proxies for (a) NVDLA-OPM and (b) BOOM-OPM.

fewer proxies is both urgent and practical, as the OPM design methodology needs to scale to larger systems. The COBIT framework ensures that both the feature and the backbone of the OPM remain lightweight, demonstrating its potential and practicality on large-scale heterogeneous platforms.

E. Hardware Implementation

1) Proxy Distribution Analysis: The distribution of selected power proxies for the two OPM designs—NVDLA-OPM and BOOM-OPM—is illustrated in Fig. 13. In the NVDLA-OPM, a large portion of proxies are located within the core computational modules: CACC (40.1%), responsible for convolution accumulation; CMAC (22.3%), handling multiply-accumulate operations; and CSC (15.7%), which performs data slicing and alignment for convolution input. These modules are known to exhibit the highest dynamic switching activities in NVDLA, driven by intensive convolution and matrix computations. The fact that our proxy selection process highlights these functional units not only ensures coverage of the most active toggling regions but also provides insight and validates the effectiveness of the method in identifying critical hotspots contributing to dynamic power.

In the BOOM-OPM implementation, the distribution shown in Fig. 13(b) reveals that the majority of proxies are concentrated in the Frontend (e.g., Fetch, ICache, BPD) and Core pipeline stages (e.g., Int. issue, Mem. issue, and FP pipeline). These modules are responsible for instruction fetch and decode, issue logic, and the execution of arithmetic and floating-point operations. Such stages typically exhibit high dynamic switching activity, particularly under complex instruction streams and pipeline stalls that are characteristic of out-of-order processors. The presence of many proxies in these regions indicates that our selection approach effectively captures the toggling behaviors associated with branch prediction, instruction-level parallelism, and floating-point computation. This confirms the relevance of the selected proxies for accurately modeling dynamic power across heterogeneous compute structures—ranging from data-parallel accelerators like NVDLA to general-purpose out-of-order CPUs like BOOM.

2) Overhead Evaluation: The optimal models selected for NVDLA and BOOM are implemented as OPMs, as explained in Section III-C4. In the algorithm model, each leaf node of the tree corresponds to a floating-point score. For hardware implementation, these scores are quantized into fixed-point data to reduce the overhead and latency of the OPM. A simple approach is to retain only the integer part of the score and represent it using a B-bit fixed-point number. This quantization process incurs a negligible loss in the accuracy of the runtime OPM. For this implementation, we set $B = 16$ to illustrate the results of the COBIT-OPM implementation. For multicycle power prediction, the additional overhead consists of a few accumulators and registers to aggregate power predictions across t/τ intervals, which is relatively inexpensive.

Under the TSMC 28-nm process, the OPM with $Q = 197$ for NVDLA operates at 800 MHz, resulting in a gate area of 10,127.25 μm^2 and a power consumption of 461.5 μW. Compared with the smaller NVDLA, the overheads are 0.49% for gate area and 0.097% for power consumption, both of which are negligible. For the BOOM CPU core, the OPM with $Q = 250$ incurs a gate area of 11910.4 μm^2 and power consumption of 569.9 μW, indicating overheads of 1.25% and 0.217%, respectively. These overheads are also negligible for the monitored hardware. Furthermore, the OPM requires only three cycles from receiving proxy toggles to generating per-cycle power predictions. First, the toggle detector takes one cycle to detect toggles. Then, each tree undergoes inference to determine a specific leaf score. Finally, it takes one cycle to pipeline the score adder.

Remarkably, the low latency of the OPM enables on-chip power management techniques at fine-grained temporal resolutions. Furthermore, with the proposed design space exploration of hyperparameters, a wide range of flexible OPM implementation options are available. This empowers power architects to leverage the toolbox effectively, allowing them to scale the methodology to a wider range of designs.

V. Conclusion

The advancements in heterogeneous platforms with ultra-high power density raise multidimensional and stringent requirements for on-chip power perception and management. In this article, we propose COBIT, a unified and intelligent OPM design and optimization framework capable of learning complex power patterns of heterogeneous architectures. By utilizing 0.028% bitwise proxies of all RTL wires in NVDLA and 0.036% in BOOM, COBIT achieves high prediction accuracy at the MAPE of 1.69% and 2.49%, and implements the OPM with a latency of just 3 cycles and small overheads. The automated design flow enables fast adaptation to other components such as memory hierarchies and interconnects. Moreover, COBIT is fully synthesizable and has been validated under the TSMC 28-nm process, making it readily deployable in practical RTL design flows. These features collectively demonstrate the practicality and scalability of COBIT for on-chip power management in heterogeneous platforms.

References

[1] (2018). *Nvidia Deep Learning Accelerator (NVDLA)*. [Online]. Available: http://nvdla.org/

[2] (2022). *Powerpro Rtl Power Estimation*. [Online]. Available: https://eda.sw.siemens.com/en-U.S./ic/powerpro/rtl-power-estimation

[3] (2023). *AMD Instinct MI300A Accelerators*. [Online]. Available: https://www.amd.com/en/products/accelerators/instinct/mi300/mi300a.html

[4] (Feb. 2023). *A Roadmap for Heterogeneous Integration for the Next Decade and Beyond an Academic Perspective*. [Online]. Available: https://r6.ieee.org/scv-eps/wp-content/uploads/sites/58/2023/02/D1-3-swami.pdf

[5] (2024). *Coremark*. [Online]. Available: https://www.eembc.org/coremark/

[6] (2024). *Nvidia Blackwell Architecture Technical Brief*. [Online]. Available: https://resources.nvidia.com/en-us-blackwell-architecture

[7] (2024). *Palladium Z1 Enterprise Emulation Platform*. [Online]. Available: https://www.cadence.com/enUS/home/tools/system-design-and-verification/acceleration-and-emulation/palladium-z1.html

[8] (2024). *Risc-v Tests*. [Online]. Available: https://github.com/riscv-software-src/riscv-tests

[9] (2024). *Sora*. [Online]. Available: https://openai.com/sora

[10] (2024). *What Is Glitch Power?*. [Online]. Available: https://www.synopsys.com/ai/what-is-glitch-power.html

[11] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 2623–2631, doi: 10.1145/3292500.3330701.

[12] A. Amid et al., "Chipyard: Integrated design, simulation, and implementation framework for custom SoCs," *IEEE Micro*, vol. 40, no. 4, pp. 10–21, Jul. 2020.

[13] Q. Bertrand, Q. Klopfenstein, P. Bannier, G. Gidel, and M. Massias, "Beyond l1: Faster and better sparse models with SKGLM," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 38950–38965. [Online]. Available: https://proceedings.neurips.cc/paperfiles/paper/2022/file/fe5c31e525e9a26a1426ab0b589f42fe-Paper-Conference.pdf

[14] J. Blank and K. Deb, "Pymoo: Multi-objective optimization in Python," *IEEE Access*, vol. 8, pp. 89497–89509, 2020.

[15] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[16] L. Cremona, W. Fornaciari, and D. Zoni, "Automatic identification and hardware implementation of a resource-constrained power model for embedded systems," *Sustain. Comput., Informat. Syst.*, vol. 29, Mar. 2021, Art. no. 100467. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2210537920301918

[17] H. David, E. Gorbatov, U. R. Hanebutte, R. Khanna, and C. Le, "RAPL: Memory power estimation and capping," in *Proc. ACM/IEEE Int. Symp. Low-Power Electron. Design (ISLPED)*, Aug. 2010, pp. 189–194.

[18] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.

[19] K. Deb and D. Kalyanmoy, *Multi-Objective Optimization Using Evolutionary Algorithms*. Hoboken, NJ, USA: Wiley, 2001.

[20] Y. Du, J. Qian, Z. Chen, and W. Shan, "An all-digital, 1.92–7.32 mV/LSB, 0.5–2 GS/s sample rate, and 0-latency prediction voltage sensor with dynamic PVT calibration for droop detection and AVS system," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 12, pp. 4888–4899, Dec. 2023.

[21] W. Godycki, C. Torng, I. Bukreyev, A. Apsel, and C. Batten, "Enabling realistic fine-grain voltage scaling with reconfigurable power distribution networks," in *Proc. 47th Annu. IEEE/ACM Int. Symp. Microarchitecture*, Dec. 2014, pp. 381–393, doi: 10.1109/MICRO.2014.52.

[22] W. Gomes et al., "Ponte Vecchio: A multi-tile 3D stacked processor for exascale computing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 42–44.

[23] M. Hao, W. Zhang, Y. Wang, G. Lu, F. Wang, and A. V. Vasilakos, "Fine-grained powercap allocation for power-constrained systems based on multi-objective machine learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1789–1801, Jul. 2021.

[24] C.-H. Hsu et al., "Adrenaline: Pinpointing and reining in tail queries with quick voltage boosting," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit.*, Sep. 2015, pp. 271–282.

[25] H. Jain and K. Deb, "An evolutionary many-objective optimization algorithm using reference-point based nondominated sorting approach, part II: Handling constraints and extending to an adaptive approach," *IEEE Trans. Evol. Comput.*, vol. 18, no. 4, pp. 602–622, Aug. 2014.

[26] M. Kang. (2023). *Heterogeneous Integration Platform for Next Generation Computing*. [Online]. Available: https://r6.ieee.org/scv-eps/wp-content/uploads/sites/58/2023/02/D2-3-kang.pdf

[27] D. Kim, J. Zhao, J. Bachrach, and K. Asanović, "Simmani: Runtime power modeling for arbitrary RTL with automatic signal selection," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2019, pp. 1050–1062, doi: 10.1145/3352460.3358322.

[28] A. K. A. Kumar, S. Al-Salamin, H. Amrouch, and A. Gerstlauer, "Machine learning-based microarchitecture-level power modeling of CPUs," *IEEE Trans. Comput.*, vol. 72, no. 4, pp. 941–956, Apr. 2023.

[29] A. W. Lewis, N.-F. Tzeng, and S. Ghosh, "Runtime energy consumption estimation for server workloads based on chaotic time-series approximation," *ACM Trans. Archit. Code Optim.*, vol. 9, no. 3, pp. 1–26, Oct. 2012, doi: 10.1145/2355585.2355588.

[30] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," *J. Mach. Learn. Res.*, vol. 18, no. 1, pp. 6765–6816, 2017.

[31] X. Li et al., "Deep reinforcement learning-based power management for chiplet-based multicore systems," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 9, pp. 1726–1739, Sep. 2024.

[32] Z. Lin, S. Sinha, and W. Zhang, "An ensemble learning approach for in-situ monitoring of FPGA dynamic power," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 9, pp. 1661–1674, Sep. 2019.

[33] Z. Lin, W. Zhang, and S. Sharad, "Decision tree based hardware power monitoring for run time dynamic power management in FPGA," in *Proc. 27th Int. Conf. Field Program. Log. Appl. (FPL)*, Sep. 2017, pp. 1–8.

[34] S. K. Mandal, G. Bhat, C. A. Patil, J. R. Doppa, P. P. Pande, and U. Y. Ogras, "Dynamic resource management of heterogeneous mobile platforms via imitation learning," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 12, pp. 2842–2854, Dec. 2019.

[35] OpenAI. (2023). *Gpt-4 Technical Report*. [Online]. Available: https://api.semanticscholar.org/CorpusID:257532815

[36] Y. Ozaki, Y. Tanigaki, S. Watanabe, and M. Onishi, "Multiobjective tree-structured Parzen estimator for computationally expensive optimization problems," in *Proc. Genetic Evol. Comput. Conf.*, Jun. 2020, pp. 533–541.

[37] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, pp. 2825–2830, 2011. [Online]. Available: http://jmlr.org/papers/v12/pedregosa11a.html

[38] H. Qiu et al., "Power-aware deep learning model serving with μ-serve," in *Proc. USENIX Annu. Tech. Conf.*, 2024, pp. 75–93.

[39] M. Sagi, N. A. V. Doan, M. Rapp, T. Wild, J. Henkel, and A. Herkersdorf, "A lightweight nonlinear methodology to accurately model multicore processor power," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 11, pp. 3152–3164, Nov. 2020.

[40] Y. Song, J. Oh, S.-Y. Cho, D.-K. Jeong, and J.-E. Park, "A fast droop-recovery event-driven digital LDO with adaptive linear/binary two-step search for voltage regulation in advanced memory," *IEEE Trans. Power Electron.*, vol. 37, no. 2, pp. 1189–1194, Feb. 2022.

[41] D. Sunwoo, G. Y. Wu, N. A. Patil, and D. Chiou, "PrEsto: An FPGA-accelerated power estimation methodology for complex systems," in *Proc. Int. Conf. Field Program. Log. Appl.*, Aug. 2010, pp. 310–317.

[42] Q. Wang, X. Mei, H. Liu, Y.-W. Leung, Z. Li, and X. Chu, "Energy-aware non-preemptive task scheduling with deadline constraint in DVFS-enabled heterogeneous clusters," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4083–4099, Dec. 2022.

[43] X. Wang, Y. Jin, S. Schmitt, and M. Olhofer, "Recent advances in Bayesian optimization," *ACM Comput. Surveys*, vol. 55, no. 13s, pp. 1–36, Jul. 2023, doi: 10.1145/3582078.

[44] Z. Xie et al., "DEEP: Developing extremely efficient runtime on-chip power meters," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2022, pp. 1–9.

[45] Z. Xie et al., "APOLLO: An automated power modeling framework for runtime power introspection in high-volume commercial microprocessors," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2021, pp. 1–14, doi: 10.1145/3466752.3480064.

[46] H. Zhang and H. Hoffmann, "PoDD: Power-capping dependent distributed applications," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Nov. 2019, pp. 1–23, doi: 10.1145/3295500.3356174.

[47] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanovic, "Sonicboom: The 3rd generation Berkeley out-of-order machine," in *Proc. 4th Workshop Comput. Archit. Res. RISC-V*, 2020.

[48] Y. Zhou, H. Ren, Y. Zhang, B. Keller, B. Khailany, and Z. Zhang, "PRIMAL: Power inference using machine learning," in *Proc. 56th Annu. Design Autom. Conf.*, Jun. 2019, pp. 1–6, doi: 10.1145/3316781.3317884.